Bukti Database & Penanganan Error — perpustakaan-api

Dokumen ini disusun dari kode dan artefak yang ada di repositori (entity, migration, Exception Filter, repository transaksi, serta test). Output/hasil eksekusi **tidak dikarang**; bila diperlukan output `psql`, hanya perintahnya yang dicantumkan.

1. Struktur tabel `books` dan constraint

Sumber: entity `src/books/entities/book.entity.ts` dan migration awal `src/migrations/1794150511860-CreateBooks.ts`. SQL `CREATE TABLE` yang dijalankan migration (baris 7 migration, tanpa diubah maknanya):

```
CREATE TABLE "books" (
  "id"        SERIAL NOT NULL,
  "isbn"      character varying NOT NULL,
  "title"     character varying NOT NULL,
  "author"    character varying NOT NULL,
  "year"      integer,
  "category"  character varying,
  "stock"     integer NOT NULL DEFAULT '0',
  "available" integer NOT NULL DEFAULT '1',
  CONSTRAINT "UQ_54337dc30d9bb2c3fadebc69094" UNIQUE ("isbn"),
  CONSTRAINT "PK_f3f2f25a099d24e12545b70b022" PRIMARY KEY ("id")
);
```

Kolom	Tipe	Nullable	Default	Constraint	Fungsi constraint
`id`	`integer` (SERIAL)	NOT NULL	sequence (`books_id_seq`)	`PRIMARY KEY (id)` (`PK_f3f2f25a099d24e12545b70b022`)	Mengidentifikasi tiap baris secara unik; menjadi acuan pencarian/pembaruan/penghapusan (`/books/:id`).
`isbn`	`character varying`	NOT NULL	—	`UNIQUE (isbn)` (`UQ_54337dc30d9bb2c3fadebc69094`)	Menjamin satu ISBN hanya milik satu buku; pelanggaran memicu error PostgreSQL kode `23505` (ditangkap aplikasi → 409). `NOT NULL` memastikan ISBN selalu terisi.
`title`	`character varying`	NOT NULL	—	`NOT NULL`	Judul wajib ada; mencegah data buku tanpa identitas judul.
`author`	`character varying`	NOT NULL	—	`NOT NULL`	Penulis wajib ada; selaras dengan validasi DTO.
`year`	`integer`	NULL	—	—	Tahun terbit bersifat opsional.
`category`	`character varying`	NULL	—	—	Kategori bersifat opsional.
`stock`	`integer`	NOT NULL	`0`	`NOT NULL` + `DEFAULT 0`	Jumlah stok selalu punya nilai numerik; jika tidak diisi, aman bernilai 0.
`available`	`integer`	NOT NULL	`1`	`NOT NULL` + `DEFAULT 1`	Status/ketersediaan numerik selalu terisi; default 1 bila tidak diisi.

Catatan: migration juga men-set sequence `books_id_seq` ke 109 (`SELECT setval(pg_get_serial_sequence('books','id'), 109, false)`) dan menyisipkan 2 baris seed (ISBN `978-602-123-456-7` dan `978-602-234-567-8`). Constraint `NOT NULL`/`UNIQUE`/`PRIMARY KEY` diberlakukan oleh PostgreSQL (lapisan database), sementara DTO (`src/books/dto/create-book.dto.ts`) memberi validasi dini di lapisan aplikasi.

Perintah `psql` untuk melihat struktur & isi (jalankan manual; password tidak ditulis di sini):

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -d perpustakaan -c "\d books"
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -d perpustakaan -c "SELECT *
FROM books ORDER BY id;"
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -d perpustakaan -c "SELECT *
FROM migrations;"
```

2. Tabel `migrations` dan file migration

Tabel `migrations` dibuat otomatis oleh TypeORM saat `migration:run` pertama (CLI TypeORM v1 mencatat eksekusi di sana). Kolomnya mengikuti bawaan TypeORM: `id` (PK), `timestamp` (bigint), `name` (varchar). Fungsinya: melacak migration mana yang sudah dijalankan agar tidak dieksekusi ulang.

Nama file migration yang dipakai (dibaca dari folder `src/migrations`):

- `src/migrations/1794150511860-CreateBooks.ts` — class `CreateBooks1794150511860` (`name = 'CreateBooks1794150511860'`). `up()` membuat tabel `books` (+ constraint), menyisipkan 2 seed, dan men-set sequence; `down()` melakukan `DROP TABLE "books"`.

3. Contoh response 400, 404, dan 409

Semua error diformat seragam oleh Exception Filter global `src/common/filters/http-exception.filter.ts` (didaftarkan di `src/main.ts`) menjadi `{ statusCode, error, message, path, timestamp }`. `message` dari validasi (array) diratakan menjadi string (`join('; ')`); error tak terduga dibalas 500 tanpa detail internal.

Kutipan inti filter (`src/common/filters/http-exception.filter.ts`):

```ts
@Catch()
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost): void {
    // ... ambil response, request, timestamp, path ...
    if (exception instanceof HttpException) {
      statusCode = exception.getStatus();
      // ... ambil error & message dari exception.getResponse() ...
      // ValidationPipe mengirim message berupa array; diratakan jadi string.
      if (Array.isArray(raw)) { message = raw.join('; '); }
      else if (typeof raw === 'string') { message = raw; }
      else { message = exception.message; }
    } else {
      statusCode = HttpStatus.INTERNAL_SERVER_ERROR;
      error = this.statusText(statusCode);
      message = 'Terjadi kesalahan pada server';
    }
    response.status(statusCode).json({ statusCode, error, message, path, timestamp });
  }
}
```

Contoh berikut **diambil dari kode/test** yang memicu error tersebut (bukan output rekaan):

- **400 — validasi body gagal** (field wajib kosong / melanggar batas). Dipicu oleh `ValidationPipe` + `CreateBookDto` (`src/books/dto/create-book.dto.ts`, `title/author/isbn` `@IsNotEmpty`, `year` `@Min(1000)`/`@Max(2100)`). Skenario di `test/books-failure.e2e-spec.ts` (kasus 1): `POST /books` body `{ title: '', author: 'Penulis', isbn: '978-1-000' }` → `expect(400)`, `body.statusCode === 400`, `body.error === 'Bad Request'`, `body.message` mengandung `title wajib diisi`, serta ada `path /books` dan `timestamp`.
- **400 — `:id` tidak valid**. Dipicu oleh `ParseIntPipe` di controller. Skenario di `test/books-failure.e2e-spec.ts` (kasus 2): `GET /books/abc` → `expect(400)`, `body.statusCode === 400`, `body.error === 'Bad Request'`.
- **404 — data tidak ditemukan**. Dipicu oleh `NotFoundException` di service. Skenario di `test/books-failure.e2e-spec.ts` (kasus 3): `GET /books/99999` → `body toMatchObject({ statusCode: 404, error: 'Not Found', path: '/books/99999' })` dan `message` mengandung `99999`.
- **409 — ISBN duplikat**. Pelanggaran UNIQUE (`23505`) ditangkap di repository lalu dilempar sebagai `ConflictException('ISBN sudah digunakan oleh buku lain')`. Skenario di `test/books-failure.e2e-spec.ts` (kasus 4): buat buku ISBN `978-409-0001`,

lalu POST ISBN yang sama → `expect(409)` dengan `body toMatchObject({ statusCode: 409, error: 'Conflict', message: 'ISBN sudah digunakan oleh buku lain', path: '/books' })`. Kasus 5 (bulk duplikat) menghasilkan 409 yang sama pada `path: '/books/bulk'`.

4. Transaksi `POST /books/bulk` — titik COMMIT dan ROLLBACK

Endpoint: `POST /books/bulk` (`src/books/books.controller.ts`, didefinisikan sebelum route `:id` agar path `bulk` tidak tertangkap sebagai parameter). Body divalidasi per elemen dengan `ParseArrayPipe({ items: CreateBookDto, whitelist: true })`. Logika simpan atomik ada di `src/books/books.repository.ts` method `createBulk`, memakai `this.dataSource.transaction(...)`.

Kutipan kode (`src/books/books.repository.ts`, `createBulk`):

```
async createBulk(data: CreateBookData[]): Promise<Book[]> {
  return this.dataSource.transaction(async (manager) => {
    const repo = manager.getRepository(Book);
    const saved: Book[] = [];
    try {
      for (const item of data) {
        const book = repo.create(item);
        saved.push(await repo.save(book));
      }
    } catch (error) {
      if (error instanceof QueryFailedError) {
        const code = (error.driverError as { code?: string } | undefined)?.code;
        if (code === UNIQUE_VIOLATION) {
          // TITIK ROLLBACK (otomatis): exception dilempar dari callback
          // transaksi → TypeORM melakukan ROLLBACK, tidak ada satu pun
          // buku dari batch ini yang tersimpan.
          throw new ConflictException('ISBN sudah digunakan oleh buku lain');
        }
      }
      // TITIK ROLLBACK (otomatis): error lain juga memicu ROLLBACK oleh TypeORM.
      throw error;
    }
    // TITIK COMMIT (otomatis): callback selesai tanpa error → TypeORM
    // melakukan COMMIT, seluruh buku batch tersimpan secara atomik.
    return saved;
  });
}
```

- **Titik COMMIT (otomatis, akhir callback — `return saved`)**: callback transaksi selesai *tanpa* melempar exception → `dataSource.transaction` menjalankan `COMMIT`, sehingga semua baris yang disimpan di dalam loop tersimpan secara atomik (response 201).
- **Titik ROLLBACK (otomatis, saat exception dilempar dari callback — blok `catch` → `throw`)**: jika salah satu `repo.save` gagal (mis. unique violation/ISBN duplikat, kode `23505`) atau error lain terjadi, exception dilempar keluar callback → `dataSource.transaction` menjalankan `ROLLBACK`. Efeknya seluruh batch batal (tidak ada baris parsial). Kegagalan `23505` dipetakan ke `ConflictException` (409) agar konsisten dengan format filter.

Bukti perilaku rollback diverifikasi di `test/books-failure.e2e-spec.ts` (kasus 5): kirim bulk berisi item valid lalu item dengan ISBN duplikat dalam batch yang sama → respons 409, lalu `dataSource.getRepository(Book).find()` mengembalikan `[]` (tidak ada yang tersimpan).

5. Ringkasan test kondisi gagal

Sumber: `test/books-failure.e2e-spec.ts` (semua memakai `perpustakaan_test` via `test/jest-setup.ts`, migration dijalankan, tabel di-`TRUNCATE` sebelum tiap test).

Skenario	Status diharapkan	File test
POST `/books` dengan `title` kosong	400 Bad Request	`test/books-failure.e2e-spec.ts` (kasus 1)

Skenario	Status diharapkan	File test
GET `/books/abc` (`:id` tidak valid)	400 Bad Request	`test/books-failure.e2e-spec.ts` (kasus 2)
GET `/books/99999` (data tidak ada)	404 Not Found	`test/books-failure.e2e-spec.ts` (kasus 3)
POST `/books` dengan ISBN yang sudah ada	409 Conflict	`test/books-failure.e2e-spec.ts` (kasus 4)
POST `/books/bulk` dengan satu ISBN duplikat → rollback (batch tidak tersimpan)	409 Conflict + tabel tetap kosong (bukti rollback)	`test/books-failure.e2e-spec.ts` (kasus 5)

Catatan: test unit (`src/**/*.spec.ts`) dan e2e lain (`test/app.e2e-spec.ts`, `test/books.e2e-spec.ts`) juga ada; tabel di atas merangkum khusus skenario kondisi gagal sesuai permintaan.

6. Screenshot bukti (folder `screenshot/`)

Berdasarkan isi folder `screenshot/` di repositori (nama file = fakta; keterangan mengikuti nama filenya):

File	Keterangan singkat
`crud-01-get-daftar-200.png`	GET daftar buku — 200
`crud-02-get-detail-200.png`	GET detail buku — 200
`crud-03-post-201.png`	POST buku — 201
`crud-04-put-200.png`	PUT buku — 200
`crud-05-patch-200.png`	PATCH buku — 200
`crud-06-delete-204.png`	DELETE buku — 204
`crud-07-get-404.png`	GET buku tidak ada — 404
`crud-08-post-400.png`	POST tidak valid — 400
`t6-error-400.png`	Error validasi — 400
`t6-error-400-id.png`	Error id tidak valid — 400
`t6-error-404.png`	Data tidak ditemukan — 404
`t6-error-409.png`	ISBN duplikat — 409
`t6-bulk-sukses.png`	POST `/books/bulk` sukses
`t6-bulk-sukses-get.png`	Verifikasi data setelah bulk sukses
`t6-bulk-rollback.png`	POST `/books/bulk` gagal (duplikat) — 409
`t6-bulk-rollback-get.png`	Verifikasi rollback (data batch tidak tersimpan)
`t6-hasil-test.png`	Hasil eksekusi test (T6)
`t6-isi-tabel-dan-migrations.png`	Isi tabel `books` dan `migrations`
`hasil-test-unit.png`	Hasil unit test
`hasil-test-e2e.png`	Hasil E2E test
`struktur-project.png`	Struktur project